COM6103 Team Software Project Report

Team	1
Project title	eWaste

Member	**Main responsibility**
Dibing Bai	Documentation and part of backend development
Ziwen Li	Database design and database implementation
Yongqiang Liu	Frontend development
Yaqun Ma	Frontend development
Xuhao Zhou	Backend development

The University of Sheffield

Contents

1 Introduction / Background to the Project

Electronic waste, or eWaste, is an increasingly common issue for individual consumers. Many people keep old electronic devices such as smartphones, laptops, tablets, games consoles and other digital equipment at home, even when those devices are no longer used. These devices may still have resale value, reusable parts or recyclable materials. However, owners often delay recycling because they are concerned about personal data, want to keep the device as a backup, still have photos or files stored on it, or believe that the device may become collectable in the future.

The client brief for this project proposed a business-to-consumer eWaste Recycling Hub. The aim was not simply to create a general recycling website, but to design a web application that helps owners make informed decisions about unwanted devices. Depending on the type, condition and demand for a device, the system should guide the owner towards an appropriate next step, such as resale through a third party, recycling, data retrieval or data wiping. The project also needed to support internal eWaste staff by giving them tools to review unknown devices, update device records and monitor operational information.

The main problem addressed by the project is the gap between a device owner's intention to dispose of old technology and the practical steps required to do so safely. A user may want to recycle a device but may not know whether it still has value, whether it should be treated as rare, or whether data can be recovered before the device is wiped. The system therefore acts as a central hub between device owners, eWaste staff and potential third-party partners. It aims to reduce uncertainty for owners while also giving the organisation a structured workflow for handling submitted devices.

The overall objective of our project was to develop a functional full-stack web application that demonstrates the core eWaste workflow. The system allows users to register and log in, submit details of their devices, receive classification-specific outcomes, and interact with related services such as data retrieval, reward or referral options and device status tracking. For staff users, the system provides tools to manage submissions, review unknown devices and update classifications. For administrators, the system includes user and role management features, as well as reporting foundations for monitoring payments, device processing and referral information.

The system was developed using a Flask backend with SQLAlchemy for database modelling and data management. The frontend was implemented using React, Vite and TypeScript to provide a modern web interface for owner, staff and administrator pages. A relational database was used to store users, devices, collection requests, retrieval requests, payment records, referral information and staff processing records. This architecture allowed the team to separate frontend interface development from backend business logic, while still supporting the main workflows required by the client brief.

Overall, the project provides a practical foundation for an eWaste Recycling Hub. While some advanced integrations were left as future work, the delivered system demonstrates the main concept of helping owners understand the status and options for their devices, while giving eWaste staff and administrators the tools needed to manage the process in a structured and traceable way.

2 Changes to Project Scope and Objectives

The original client brief required a business-to-consumer eWaste Recycling Hub where device owners could submit unwanted electronic devices and receive suitable next steps based on classification. The expected system included account registration and login, device classification, resale/referral support, recycling and data retrieval workflows, staff management tools, admin user management, payment support and operational reporting.

The overall objective of the project did not change: the team still aimed to build a working web application for device owners, staff and administrators. However, during sprint planning, client discussions and TA feedback, the team refined the scope to focus on the most important and demonstrable workflows. Some advanced features were simplified or moved to future work in order to keep the final system stable and realistic.

Original requirement	Final scope / change	Reason
Desktop usage was essential, with mobile browser support as an option.	The final system focused on desktop browser usage.	Desktop was the core requirement, so mobile optimisation was deprioritised.
Current and rare devices should support third-party resale and QR/referral options.	Referral and reward foundations were implemented, but the full QR/resale owner journey was left as future work.	The complete owner-facing QR and resale flow required more time and polish.
Recycle devices should support data retrieval and data wiping.	Data retrieval and staff wipe-job foundations were implemented, but production email/cloud delivery was not fully completed.	Secure file delivery and real external service setup added extra complexity.
PayPal and Stripe sandbox payments were expected.	Payment provider support was added, but full end-to-end sandbox callback verification still needs further confirmation.	External payment callbacks required stable environment configuration and testing time.
A wide set of client features were initially treated as important.	The team prioritised authentication, device submission, classification, staff review, admin management, retrieval and reporting foundations.	This reduced late-stage risk and allowed the team to deliver a more stable system.

These changes were made to control project scope rather than reduce the purpose of the system. The team focused on delivering the core client value: helping owners understand what can happen to their devices, while giving staff and administrators tools to manage the process. Features depending heavily on external integrations, such as fully polished QR handling, production email/cloud delivery and fully verified payment callbacks, were treated as future work instead of being presented as fully complete.

3 User Stories

The following user stories were created from the original client brief and refined during sprint planning. They describe the main functionality expected from the eWaste Hub from the perspective of device owners, staff members and administrators. The priorities reflect the importance of each story to the core workflow of the system, while the status shows whether the final implementation fully or partially delivered the story.

Story ID	As a...	I want to...	So that...	Priority	Status	Sprint
US1	Device owner	Register for an account and log in securely	I can access my personal eWaste account and submit device information	High	Complete	Sprint 1–2
US2	Device owner	Submit details of an unwanted electronic device	The system can assess the device and suggest an appropriate next step	High	Complete	Sprint 1–2
US3	Device owner	See a classification-specific result for my device	I know whether the device should be resold, recycled, reviewed, or treated as rare	High	Complete / Partial	Sprint 2
US4	Device owner	View the status of my submitted devices	I can track whether my device is pending, reviewed, or being processed	High	Complete	Sprint 2
US5	Device owner	Request paid data retrieval before recycling	I can recover important files before the device is wiped or recycled	High	Partial to Complete	Sprint 2–3
US6	Device owner	Receive referral or reward options for current or rare devices	I can understand possible resale or bonus options before handing in the device	Medium	Partial	Sprint 3
US7	Staff member	Review unknown or newly submitted devices	The device database can be updated and the owner can receive a suitable outcome	High	Complete	Sprint 1–2
US8	Staff member	Create and update device records, including classification and visibility	The system can keep device information accurate and avoid showing incomplete records	High	Complete	Sprint 2

US9	Staff member	Manage data retrieval and data wiping jobs	Devices can be processed safely before reuse or recycling	Medium	Partial	Sprint 3
US10	Administrator	Manage user accounts and change user roles	Staff access can be controlled and users can be managed safely	High	Complete	Sprint 2–3
US11	Staff / Administrator	View reports about payments, device processing and referral activity	The organisation can monitor operational progress and business activity	Medium	Partial	Sprint 3

These user stories show that the project focused first on the core owner journey: account access, device submission, classification and status tracking. Staff and administrator stories were then added to support the internal workflow required by the client brief, including device review, role management and reporting. Some stories, especially those involving external integrations such as QR referral handling, production email/cloud delivery and fully verified payment callbacks, were only partially completed and are discussed further in the uncompleted features and future work sections.

4 Analysis & Design

This section describes the main analysis and design decisions made during the development of the eWaste Hub. The design was based on the original client brief, sprint planning, team discussions and feedback received during the project. The system was designed to support three main user groups: device owners, staff members and administrators. It also needed to support both public-facing workflows, such as device submission and classification, and internal workflows, such as staff review, device management, data retrieval and reporting.

4.1 Requirements Analysis

The main purpose of the system was to provide a web hub where device owners could submit details of unwanted electronic devices and receive appropriate guidance based on the device classification. The client brief required the system to distinguish between different device outcomes, such as current, recycle, rare, unknown or unwanted devices. This meant that the system needed to do more than simply store device records; it also needed to provide a structured workflow from owner submission to staff processing.

The team identified three main user roles. A device owner can register, log in, submit device details, view classification results and track the status of submitted devices. A staff member can review unknown devices, manage device records, update classifications and support operational processes such as retrieval and wipe jobs. An administrator can manage users, update staff access and view higher-level reports.

The key functional requirements identified for the system were:

Requirement area	**Description**
Authentication	Users should be able to register, log in and access protected pages.
Device submission	Owners should be able to submit details of unwanted devices.
Device classification	Devices should be classified as current, recycle, rare, unknown or unwanted.
Owner workflow	Owners should receive suitable next steps based on classification.
Staff workflow	Staff should review requests, manage devices and process unknown devices.
Admin workflow	Admins should manage users, roles and reporting information.
Data retrieval / wiping	The system should support the data retrieval and wipe-job process.
Reporting	Staff or admins should be able to view payment, processing and referral information.

Several non-functional requirements were also considered. The system needed to be usable for non-technical users, so the owner-facing pages were designed to be clear and task-focused. Security was also important because the system stores user accounts, device requests and processing records. For this reason, authentication, protected routes and role-based access control were included in the design. Maintainability was considered by separating frontend components, backend routes, database models and business logic into different parts of the project.

4.2 System Architecture

The eWaste Hub was designed as a full-stack web application with a separate frontend, backend and database layer. The frontend was implemented using React, Vite and TypeScript. It provides separate interfaces for device owners, staff members and administrators. The backend was implemented using Flask and provides REST API endpoints for authentication, device management, requests, retrieval workflows, referrals, payments and reports. SQLAlchemy was used to model and interact with the relational database.

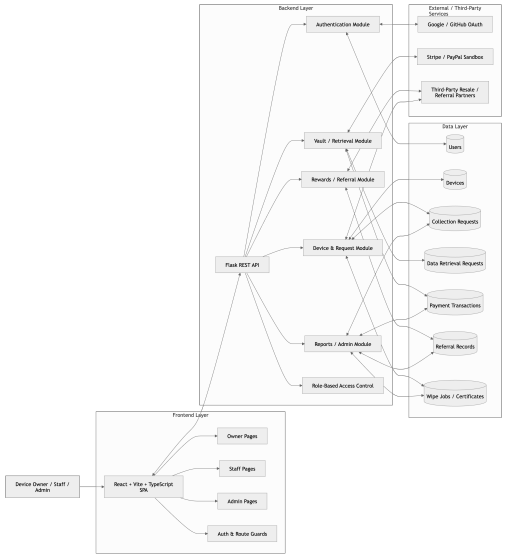

Figure 1: Overall System Architecture Diagram

The system follows a three-layer architecture consisting of a React frontend, Flask backend and relational database. The frontend sends API requests to the backend, while the backend handles authentication, permissions, business logic and database access. External services such as Google/GitHub OAuth, Stripe/PayPal sandbox payment providers and third-party resale/referral partners are represented as supporting integrations.

This architecture was chosen because it allowed the team to divide work clearly between frontend, backend and database development. Frontend developers could focus on user interface and workflow design, while backend developers could focus on validation, permissions, database interaction and API logic. The separation also made the system easier to test and maintain, because backend functionality could be tested independently from the frontend interface.

The basic system flow is:

1. The user interacts with the React frontend.
2. The frontend sends requests to the Flask backend API.
3. The backend validates input and checks authentication or role permissions.
4. SQLAlchemy is used to read or update database records.
5. The backend returns a response to the frontend.
6. The frontend displays the updated result to the correct user role.

4.3 UML / Design Diagrams

The use case design focused on the three main actors in the system: device owners, staff members and administrators. Each actor was given access to different features based on their responsibilities. This helped ensure that public users could only access their own device workflows, while staff and administrators could access internal management features.

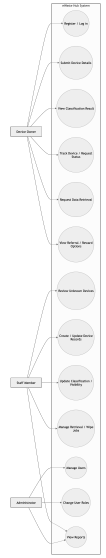

Figure 2: Use Case Diagram

The use case diagram shows the main functions available to each role. Device owners can register, log in, submit device details, view classification results, track request status and request data retrieval. Staff members can review unknown devices, manage device records, update classifications and manage retrieval or wipe jobs. Administrators can manage users, change roles and view reports.

The most important workflow in the system is the device submission and classification process. This workflow begins when a device owner submits information about an unwanted device. The backend validates the submitted data and creates the necessary request records. The system then applies classification logic to decide whether the device is current, recyclable, rare, unknown or unwanted. If the system can classify the device, the owner is shown the relevant next step. If the device is unknown, it is placed into a staff review workflow.

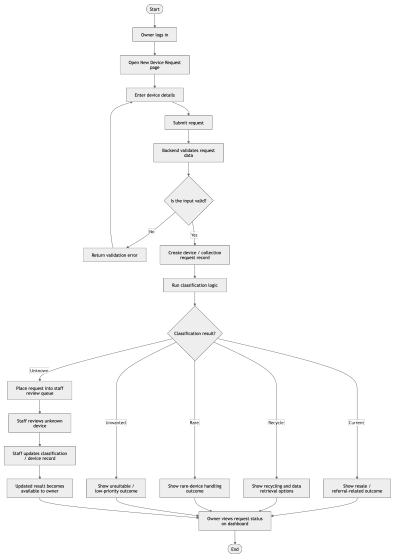

Figure 3: Device Submission and Classification Activity Diagram

The diagram shows how an owner submits device information, how the backend validates and classifies the request, and how different outcomes lead to different workflows. Current devices may lead to resale or referral-related options, recycle devices may lead to recycling or data retrieval, rare devices may require special handling, and unknown devices are sent to staff for review.

This workflow was central to the project because it connects the owner-facing side of the application with the internal staff process. It also prevents uncertain devices from being handled incorrectly. Instead of giving an unreliable result, unknown devices can be reviewed by staff and updated in the device database.

4.4 Algorithm / Database Design

The database was designed around the lifecycle of a device request. The most important entities are users, devices and collection requests. Additional entities support data retrieval, payments, referrals, wipe jobs and reporting. This design allows the system to track a device from initial owner submission through classification, processing and follow-up actions.

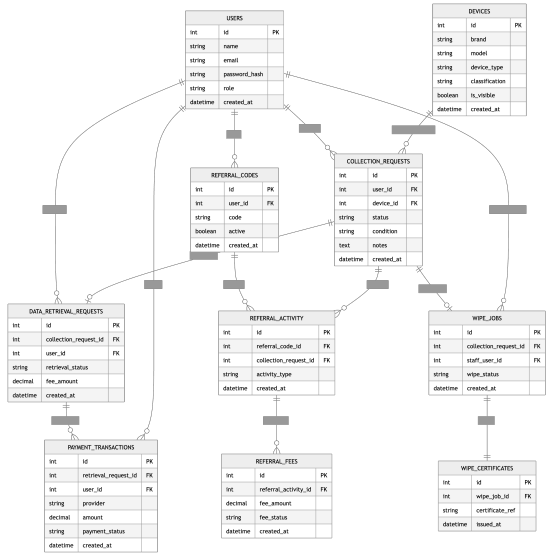

Figure 4: Entity–Relationship Diagram

The database is centred around users, devices and collection requests. A user can submit multiple collection requests, and each request is linked to a device. Depending on the outcome, a request may also be linked to data retrieval requests, payment transactions, referral records or wipe jobs. This structure supports both the public owner workflow and the internal staff/admin workflow.

The simplified classification process is:

1. The owner submits device details.
2. The backend validates the required information.
3. The system compares the submitted information with known device records and classification rules.
4. The device is assigned a classification such as current, recycle, rare, unknown or unwanted.
5. The classification determines the next option shown to the owner.
6. If the device cannot be classified confidently, it is placed into the staff review queue.

This approach was chosen because it keeps the owner workflow simple while still allowing staff intervention when necessary. It also makes the system extensible. More detailed classification rules, external pricing information or additional third-party resale integrations could be added in the future without redesigning the full system.

The database design also supports reporting. Since payment transactions, referral information, collection requests and wipe-job records are stored as structured data, the system can generate operational information for staff and administrators. Although some advanced integrations were left as future work, the current design provides a strong foundation for extending the eWaste Hub in later sprints.

5 Test Plan, Test Documentation and Test Results

Testing was an important part of the project because the eWaste Hub includes several connected workflows, including authentication, device submission, classification, staff processing, admin role management, data retrieval, payments and referral foundations. The team used a combination of automated backend tests, frontend build and lint checks, and manual workflow testing to check that the system behaved correctly from both a technical and user perspective.

5.1 Testing Strategy

The testing strategy was divided into three main areas. First, backend automated tests were used to check API routes, authentication, permissions, database operations and important workflow logic. This was important because most of the business rules, such as role-based access and request processing, are handled by the Flask backend.

Second, frontend build and lint checks were used to confirm that the React and TypeScript frontend could compile successfully and did not contain major syntax or linting errors. Although no dedicated automated frontend test suite was found, the build and lint checks provided basic evidence that the frontend code was structurally valid.

Third, manual testing was used to check complete user workflows through the browser. These manual tests focused on whether device owners, staff members and administrators could use the system as intended. This was necessary because some workflows, such as submitting a device and checking the result on the dashboard, involve both frontend and backend behaviour.

5.2 Test Plan

The test plan focused on the main user journeys and the highest-risk parts of the system. Authentication and role-based access were tested first because they protect the rest of the application. Device submission and classification were then tested because they represent the core client workflow. Staff and admin functions were tested to ensure that internal users could manage devices, users and reports correctly.

Test area	Purpose	Testing method
Authentication	Check that users can register, log in and access protected pages	Automated backend tests and manual browser testing
Role-based access	Check that owner, staff and admin users can only access suitable pages	Automated backend tests and manual testing
Device submission	Check that owners can submit device details successfully	Automated backend tests and manual browser testing
Classification	Check that submitted devices receive suitable classification outcomes	Backend logic tests and manual workflow testing
Staff workflow	Check that staff can review unknown devices and update device records	Manual testing and backend route tests
Admin workflow	Check that admins can manage users and roles	Automated backend tests and manual testing
Data retrieval	Check that retrieval requests, payment foundations and download links are handled	Backend tests and manual testing
Referral / rewards	Check that referral and reward foundations are available	Backend tests and manual testing
Reporting	Check that admin/staff report pages and report APIs display relevant data	Manual testing and backend API tests
Deployment / build	Check that the system can be built and started successfully	Frontend build, lint and launcher testing

5.3 Sample Test Cases

The following sample test cases show how the main features were tested. These are representative tests rather than a full list of every backend test.

Test ID	Feature	Steps	Expected result	Actual result	Status
TC01	User registration and login	Register a new account, then log in using valid credentials	User is authenticated and can access protected owner pages	Login workflow worked and JWT-based access was available	Pass
TC02	Device submission	Log in as an owner and submit a new device request with valid device details	Device request is created and stored in the system	Request was created and shown in the owner workflow	Pass
TC03	Device classification	Submit devices with different type, age and condition values	System returns an appropriate classification such as current, recycle, rare or unknown	Classification logic returned classification-specific outcomes	Pass
TC04	Owner dashboard	Log in as an owner and open the dashboard	Owner can view submitted devices and request status	Dashboard displayed request and status information	Pass
TC05	Staff unknown queue	Log in as staff and review an unknown device	Staff can update the device record or classification	Staff review workflow was available	Pass
TC06	Admin role management	Log in as admin and change a user role	User role is updated and protected access changes accordingly	Admin role management was implemented	Pass
TC07	Data retrieval workflow	Create a retrieval request and start the payment/download process	Retrieval lifecycle is created and visible to the user	Retrieval foundation worked, but production email/cloud delivery was not fully completed	Partial
TC08	Referral / reward flow	Open reward or referral-related pages for suitable devices	User can view referral or reward information	Referral foundation was implemented, but the complete QR/resale owner journey was incomplete	Partial
TC09	Frontend build	Run the frontend build command	Frontend compiles without build failure	Build completed successfully, with a bundle size warning	Pass
TC10	Frontend lint	Run the frontend lint command	No major lint errors are reported	Lint completed with no reported diagnostics	Pass

5.4 Automated Testing Evidence

The strongest automated testing evidence came from the backend test suite. The backend tests were run using Python unittest discovery. These tests covered API and integration-style behaviour using temporary SQLite databases and mocked external authentication providers where needed.

Check	Result	Evidence / command
Backend unit / integration tests	95 tests passed in 9.543s	backend/.venv/bin/python -m unittest discover -s backend/tests -v
Backend pytest command	Did not run because pytest was not installed in the backend virtual environment	pytest returned "No module named pytest"
Frontend build	Passed	npm run build
Frontend lint	Completed with no reported diagnostics	npm run lint
Frontend automated test suite	Not found	frontend/package.json did not include a dedicated test script

5.5 Test Results and Reflection

Overall, the testing results showed that the main system foundations were working. Authentication, user roles, device submission, classification, staff device management, admin role management, retrieval foundations, referral foundations and reporting APIs were supported by either automated tests, manual tests or both. The backend test suite was particularly useful because it allowed the team to check important business logic repeatedly after changes.

Testing also helped identify several limitations. Some advanced features were only partially completed, especially the full QR/resale owner journey, production email/cloud delivery, fully verified PayPal/Stripe sandbox callbacks and the owner-side data wipe flow. Frontend automated testing was also a gap, as the frontend relied mainly on build checks, linting and manual testing rather than a full automated UI test suite.

The team responded to these risks by focusing the final sprint on stabilisation rather than adding many new features. This helped prevent late-stage defects and made the final system more reliable for demonstration. If the project continued into another sprint, the main testing improvements would be to add automated frontend tests, expand end-to-end workflow testing, verify live payment sandbox callbacks and add more test evidence for email/cloud download behaviour.

6 Team Management & Communication

Team management and communication were important throughout the project because the eWaste Hub required frontend development, backend development, database design, testing, documentation and regular client/adviser communication. The team organised work by dividing responsibilities across the main parts of the system, while still keeping members involved in shared planning, sprint reviews and integration discussions.

6.1 Team Organisation

The team was organised around the main technical and documentation areas of the project. This allowed different parts of the system to be developed in parallel. For example, frontend pages could be designed while backend APIs and database models were still being implemented and refined. The main responsibilities were divided as follows:

Team member	Main responsibilities
Dibing Bai	Documentation, report preparation and part of backend development
Ziwen Li	Database design, database setup and database-related implementation
Yongqiang Liu	Frontend development and user interface implementation
Yaqun Ma	Frontend development and user interface implementation
Xuhao Zhou	Backend development, API implementation and backend integration

This division helped the team make progress across different areas at the same time. Backend members focused on Flask APIs, authentication, role-based permissions and business logic. Frontend members focused on React pages, user workflows and connecting the interface to backend endpoints. The database work supported both sides by defining the structure for users, devices, requests, payments, referrals and processing records. Documentation work was also maintained throughout the project so that sprint decisions, meeting records and report evidence were not left until the final stage.

Although members had main areas of responsibility, the project still required collaboration. For example, frontend and backend members had to communicate when API responses changed, and database changes had to be understood by both backend and testing work. This helped reduce misunderstandings during integration.

6.2 Tools and Workflow

The team used GitLab as the main tool for code management. Git allowed members to commit changes, manage versions and reduce the risk of losing work. Branches and merge requests were used to organise development work, especially when different members were working on separate parts of the system. The repository also provided evidence of project progress through commits, file changes and feature development history.

The main development technologies were React, Vite and TypeScript for the frontend, Flask for the backend, and SQLAlchemy with a relational database for data storage. These technologies supported a clear separation between the user interface, backend API logic and database layer. This structure made it easier to assign tasks and allowed team members to work on different layers of the system.

In addition to code management, the team used sprint documents and meeting records to plan work. Sprint documents helped the team identify priorities, record what had been completed and decide what should be moved to the next sprint. Meeting records were used to track decisions, problems and action points. This was useful because it gave the team a written record of how the project changed over time.

6.3 Communication

Regular communication was necessary because the project involved several connected workflows, including device submission, classification, staff review, admin management, retrieval, referral and reporting. Inside the team, meetings were used to discuss progress, current problems and task allocation. When a member was blocked, the team could discuss the problem and decide whether another member could help or whether the task needed to be adjusted.

The team also communicated with the TA and client during the project. These meetings helped the team understand feedback more clearly and check whether the system was still aligned with the client brief. For example, feedback and sprint discussions helped the team focus on core workflows rather than trying to complete too many advanced features at the same time. This communication was important for controlling scope and improving the final system.

Communication was especially important during integration. Frontend pages depended on backend API behaviour, and backend logic depended on the database structure. Therefore, the team had to share updates when endpoints, data fields or workflows changed. This helped reduce integration problems and made it easier to prepare the system for testing and demonstration.

6.4 Support and Upskilling

Several parts of the project required team members to learn or improve skills during development. These included OAuth login, frontend-backend integration, Flask API design, database modelling, role-based access control and test setup. Because not every member had the same experience at the start, support and upskilling became an important part of the team process.

Team members supported each other by sharing setup steps, explaining bugs, discussing implementation choices and helping with unfamiliar parts of the code. For example, when frontend pages needed to connect to backend routes, frontend and backend members had to clarify request formats and expected responses. Similarly, database-related changes needed to be explained so that backend routes and frontend displays could use the correct fields.

This support helped the team solve problems more quickly and also improved members' understanding of the overall system. It reduced the risk of one person becoming the only person who understood a particular feature. This was important for a team project because the final product depended on integration between several areas rather than isolated individual work.

6.5 Contribution Balance

The team's contributions were not identical in every sprint, but the work was shared across different types of tasks. Some members contributed more to implementation, while others contributed more to database design, testing, documentation, meeting records or report preparation. This was expected because the project required more than only writing code. A successful final submission also needed sprint evidence, testing evidence, setup information, design explanation and clear documentation.

The team attempted to keep the workload balanced by assigning tasks according to the needs of each sprint and the strengths of individual members. For example, frontend tasks were mainly handled by the frontend members, while backend and database tasks were handled by members working on those areas. Documentation and report preparation were also treated as important project tasks rather than secondary work.

One challenge was that some tasks became more demanding during integration, especially when frontend pages, backend APIs and database models had to work together. To manage this, the team discussed progress regularly and adjusted work when needed. Overall, the contribution balance was reasonable because each member contributed to a necessary part of the project.

6.6 Meeting Minutes

The team kept meeting minutes and sprint records during the project. These records were used to document discussion points, decisions, progress updates and tasks for the next stage. They were useful because they allowed the team to look back at earlier decisions and understand why certain changes were made.

The project evidence includes team meeting records from multiple weeks, TA meeting notes, client meeting records and sprint documents. These records show that the team regularly reviewed progress and responded to feedback during development. In the main report, two sample meeting minutes are included to show how the team communicated and managed decisions during the project. Additional meeting records can be placed in the appendix as supporting evidence.

Meeting minutes were also useful for tracking scope changes. As the project developed, the team had to decide which features should be prioritised and which features should be moved to future work. Recording these decisions helped ensure that the final report could explain not only what was completed, but also why some advanced features were simplified or left incomplete.

Overall, the team management process was effective because the group used a clear division of responsibilities, regular communication, GitLab workflow, sprint planning and meeting records. These processes helped the team build a working system while also collecting evidence of teamwork, planning and decision-making.

7 Planned & Completed Features

This section describes the features that were planned during the project and the features completed in the final system. The project brief included a wide range of requirements, including account access, device classification, staff processing, admin management, payment support, data retrieval, referral features and reporting. Because the project had a fixed deadline, the team used sprint planning and priority levels to decide which features should be implemented first.

7.1 Sprint Plan / Prioritisation

The team prioritised features according to their importance to the client brief and the overall eWaste workflow. Features that were essential for demonstrating the system were treated as high priority. Features that improved the business process but were not required for the core workflow were treated as medium priority. More advanced integrations or polish items were treated as lower priority or future work if they could not be completed safely within the project timeframe.

Priority	Planned feature area	Reason for priority
High	User registration and login	Users needed secure access before they could submit devices or use protected pages.
High	Role-based access for owner, staff and admin	The client brief required different user roles, so access control was essential.
High	Device submission	This was the starting point of the owner workflow and the main purpose of the system.
High	Device classification	The system needed to decide whether a device was current, recycle, rare, unknown or unwanted.
High	Staff review and device management	Staff needed tools to review unknown devices and update device records.
High	Admin user and role management	Admins needed to manage users and upgrade accounts for staff access.
Medium	Data retrieval workflow	This was important to the client brief, but involved more complex workflow and payment logic.
Medium	Referral / reward foundation	This supported current and rare device handling, but the full QR flow required extra polish.
Medium	Reporting	Reports were useful for staff/admin monitoring, but depended on core records existing first.
Lower / Future work	Full production email/cloud delivery, mobile optimisation, fully polished QR journey	These features required external services, extra UI polish or more testing time.

The team's sprint plan therefore focused first on creating a usable foundation: authentication, database models, device submission, classification and role-based pages. Later work focused on staff tools, admin management, retrieval, referrals and reporting. This order helped ensure that the core system could be demonstrated even if some advanced features were not fully completed.

7.2 Completed Features

The final system completed the main foundation of the eWaste Hub. The completed features cover the core owner journey, staff workflow and administrator workflow.

Feature	Related user story	Description of completed work	Client value

Account registration and login	US1	Users can register, log in and access protected areas of the application. The system also includes support for local login and third-party authentication foundations.	Allows owners, staff and admins to use the system securely.
Role-based access control	US1, US10	The system separates access for device owners, staff members and administrators. Protected pages and backend permission checks restrict users to suitable actions.	Supports the client requirement for owner, staff and admin roles.
Device submission	US2	Device owners can submit details of unwanted devices through the owner interface. Submitted information is stored and linked to the user.	Provides the main entry point for the eWaste recycling workflow.
Device classification	US3	Submitted devices can be classified as current, recycle, rare, unknown or unwanted based on device information and system logic.	Helps owners understand what can happen to their devices.
Owner dashboard and status tracking	US4	Owners can view submitted devices and track request information through the dashboard.	Gives users visibility of their device submissions and next steps.
Staff request and device management	US7, US8	Staff members can review device requests, manage device records, update classifications and control visibility/draft status.	Allows the eWaste company to maintain accurate device data and process submissions.
Unknown device review workflow	US7	Devices that cannot be confidently classified can be placed into a staff review workflow. Staff can then review and update the record.	Prevents uncertain devices from receiving unreliable automatic outcomes.
Admin user management	US10	Administrators can view users and update roles, including upgrading users to staff where required.	Supports controlled staff access and system administration.
Data retrieval foundation	US5	The system includes a retrieval request lifecycle and payment-related foundations for data retrieval services.	Supports the client aim of offering paid data retrieval before recycling.
Referral and reward foundation	US6	The system includes referral/reward-related pages and backend foundations for partner and referral activity.	Supports resale/referral handling for current or rare devices.
Reporting foundation	US11	Staff/admin reporting APIs and report pages exist for operational data such as payments, device processing and referral activity.	Helps the organisation monitor activity and business processes.

Backend automated testing	Supports all major workflows	The backend test suite was implemented and passed 95 tests, covering API and integration-style behaviour.	Provides evidence that key backend workflows were tested and are reliable.

8 Uncompleted Features

Although the final system delivered the main eWaste Hub workflow, some features from the original client brief were not fully completed. These were mainly advanced integrations or user experience refinements that depended on external services, extra testing time or further UI polish. The team chose to be realistic about these limitations rather than present partially completed features as fully finished.

Uncompleted / partial feature	**Current status**	**Reason not fully completed**	**Next step**
Full QR referral and resale owner journey	Partial	The referral and reward foundation exists, but the complete owner-facing QR display and resale flow were not fully polished.	Complete QR generation/display, connect it to referral tracking and test the full owner journey.
Rare-device resale workflow	Partial	Rare classification is supported, but the user journey for rare devices is not as complete as the current/recycle workflows.	Add clearer rare-device outcome pages and connect rare devices to suitable third-party resale options.
Production email and cloud delivery for data retrieval	Not fully completed	Secure download foundations exist, but real email sending and production cloud storage were not fully integrated.	Add email service configuration, cloud storage integration and expiry-based download testing.
Fully verified PayPal and Stripe sandbox callbacks	Partial	Payment provider support exists, but full end-to-end sandbox callback verification still needs confirmation.	Complete sandbox callback testing and record successful payment evidence.
Owner-side data wipe choice persistence	Partial	The frontend includes data wipe-related choices, but this was not fully connected through the backend workflow.	Store owner wipe preferences in the backend and display them in staff processing pages.
Frontend automated test suite	Not completed	The frontend was checked through build, lint and manual testing, but no dedicated automated frontend test suite was implemented.	Add component or end-to-end tests for the main owner, staff and admin workflows.
Mobile optimisation	Not completed	Desktop browser usage was the essential requirement, so mobile browser optimisation was deprioritised.	Improve responsive layout and test key workflows on mobile devices.
Complete production deployment guide	Partial	The setup guide and launcher scripts exist, but there was no full production deployment guide.	Add production deployment steps, environment configuration and troubleshooting notes.

The main reason these features were not completed was scope control. The original brief included many business workflows, and some of them required external integrations such as payment providers, email services, cloud storage or third-party resale systems. These integrations introduced extra configuration and testing risks. During the final sprint, the team decided to focus on stabilising the core system instead of adding more features that might not be reliable before submission.

The uncompleted features do not prevent the system from demonstrating the central purpose of the eWaste Hub. Users can still submit devices, receive classification-related outcomes and interact with the main owner workflow. Staff and administrators can also manage device records, user roles and reporting foundations. However, the incomplete features show where the system would need more work before being used as a production service.

If the project continued into another sprint, the first priority would be to complete the QR/referral journey and verify the payment sandbox flow, because these are directly linked to the client's business model. The second priority would be to improve the secure data retrieval delivery process through real email and cloud storage integration. The third priority would be to add automated frontend or end-to-end tests so that future changes could be tested more safely.

9 Screenshots of Important / Relevant Pages

This section presents selected screenshots from the final eWaste Hub application. These screenshots are not intended to replace the user guide. Instead, they show the most important and relevant pages that demonstrate the main system workflows, including public access, owner device submission and staff processing.

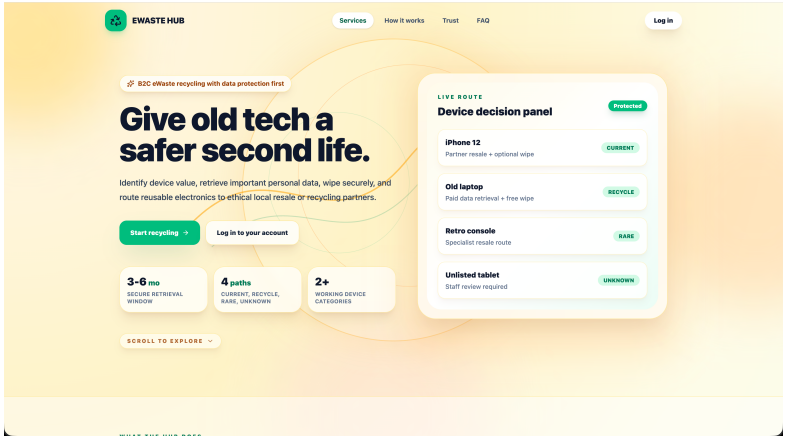

Figure 5: Public homepage and device decision concept

This screenshot shows the public homepage of the eWaste Hub. It introduces the purpose of the system with the message "Give old tech a safer second life" and explains that the hub can identify device value, support secure data retrieval and route reusable electronics to ethical resale or recycling partners. The device decision panel on the right visually summarises the main classification outcomes, including current, recycle, rare and unknown. This screenshot is useful because it communicates the system's purpose and the main client problem being addressed.

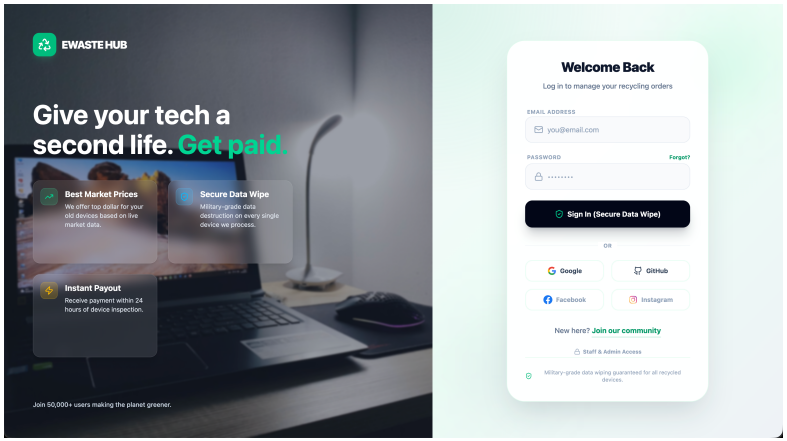

Figure 6: Login page and public access screen

This screenshot shows the login page for the eWaste Hub. It presents the system branding and explains key service values such as market prices, secure data wiping and fast payout. The login form allows users to sign in with an

email address and password, while also showing third-party login options such as Google and GitHub. This page is important because authentication is the entry point for protected owner, staff and admin workflows.

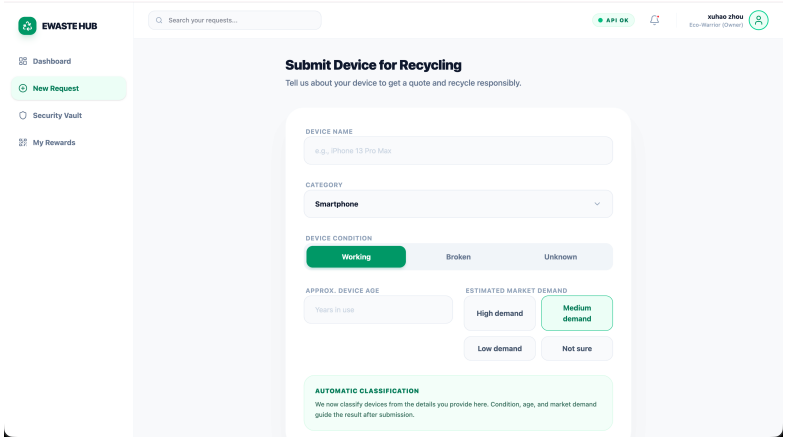

Figure 7: Owner device submission page

This screenshot shows the owner-facing page for submitting a device for recycling. The user can enter the device name, choose a device category, select the device condition, enter the approximate device age and choose an estimated market demand level. This page demonstrates the start of the main owner workflow. The information entered here is used by the system to create a device request and support automatic classification after submission.

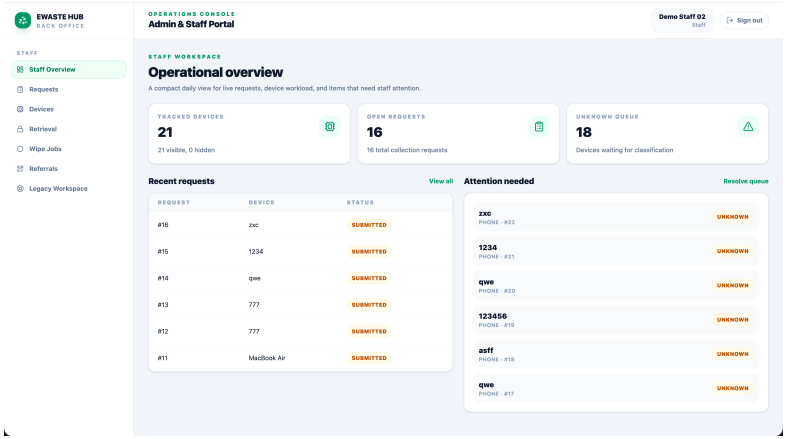

Figure 8: Staff operational overview page

This screenshot shows the staff-facing operational overview page. It displays key operational information such as tracked devices, open requests and the unknown queue. It also shows recent submitted requests and devices requiring staff attention. This page demonstrates that the system is not only designed for device owners but also supports internal eWaste staff workflows. Staff users can use this information to monitor live requests, identify devices that need classification and manage operational workload.

These screenshots demonstrate the main structure of the delivered system. The public pages explain the purpose of

the eWaste Hub and encourage users to begin the recycling process. The owner page shows how users submit device information, while the staff page shows how the organisation can monitor and process submitted devices. Together, these pages provide evidence that the system supports both customer-facing and internal operational workflows.

10 Conclusion

The eWaste Hub project gave the team practical experience of developing a full-stack web application from a client brief. The final system delivered the main foundations of a business-to-consumer eWaste Recycling Hub, including authentication, device submission, classification, role-based access, staff review, admin user management, data retrieval foundations, referral foundations and reporting support. Although some advanced integrations were not fully completed, the project successfully demonstrated the core workflow of helping device owners understand what can happen to their unwanted devices, while giving staff and administrators tools to manage the process.

10.1 What We Learned

From a technical perspective, the team learned how to build a multi-role web application using a React frontend, Flask backend and relational database. We gained experience with frontend-backend integration, SQLAlchemy data modelling, authentication, role-based access control and backend testing. The project also showed the importance of designing the database carefully, because many features depended on shared records such as users, devices, requests, payments, referrals and wipe jobs.

From a teamwork perspective, we learned that clear communication and planning are essential in a group software project. Frontend, backend, database and documentation tasks had to progress at the same time, so the team needed regular meetings, sprint records and shared updates. This helped us coordinate API changes, database changes, testing and report evidence more effectively.

10.2 Challenges and How We Resolved Them

One main challenge was the size of the original client brief. It included many workflows, such as resale, recycling, data retrieval, wiping, payments, staff reporting and admin management. To manage this, the team prioritised the core workflow first: authentication, device submission, classification, staff review and admin management. More complex features, such as QR referral handling, production email/cloud delivery and fully verified payment callbacks, were treated as future work.

Another challenge was frontend-backend integration. Frontend pages depended on backend API responses, while backend logic depended on database structure. The team resolved this through regular communication, testing and gradual integration. Near the final sprint, the team also shifted focus from adding new features to stabilisation, testing and documentation, which made the final system more reliable for demonstration.

10.3 Future Work

If the project continued into a fourth sprint, the first priority would be to complete the QR referral and resale journey. This would include displaying real QR codes to users, linking each QR code to a referral record, tracking partner interactions and making the resale journey clearer for current and rare devices. This would strengthen the business value of the system because referrals and resale partners are an important part of the eWaste Hub model.

A second priority would be to improve the data retrieval delivery process. The current system provides retrieval and secure download foundations, but a future version should integrate production cloud storage, automated email notifications, clearer expiry handling and stronger audit records for downloaded files. This would make the retrieval workflow more realistic and suitable for a production environment.

Further improvements would include adding automated frontend or end-to-end tests for the owner, staff and admin journeys, improving the user settings and notification features, strengthening mobile responsiveness, and expanding the admin analytics pages with richer charts, filters and export options. A more complete production deployment guide should also be written, covering environment variables, secure configuration, troubleshooting and public demo setup. Overall, these improvements would move the project from a strong academic prototype towards a more complete production-ready service.

Appendix

Appendix A. User Guide

This appendix provides a short user guide for ordinary users of the eWaste Hub system. It explains the main steps a user follows when accessing the application, creating an account, submitting a device and checking the result. The guide is written for non-technical users and focuses on the main functions visible in the final application.

A.1 Opening the Application

Users first open the eWaste Hub website in a browser. For local demonstration, the application normally opens at:

```
http://127.0.0.1:5173
```

The homepage introduces the purpose of the system. It explains that the platform helps users give old electronic devices a safer second life by checking device value, supporting secure data retrieval and guiding devices towards resale, reuse or recycling.

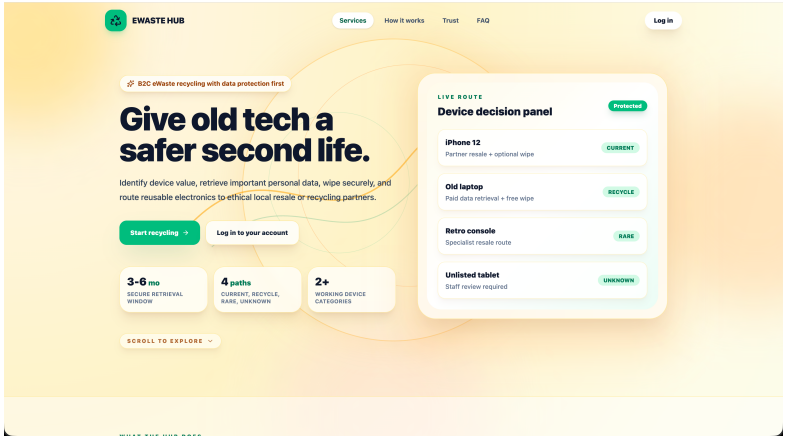

Figure 9: Homepage and eWaste Hub introduction

A.2 Creating an Account or Logging In

To use protected functions, users need to log in. A new user can select the registration option and enter their name, email address and password. Existing users can log in with their email address and password.

The system also presents third-party login options. Facebook and Instagram are available as demonstration login flows, while Google and GitHub require external OAuth configuration before they can be used as real login providers.

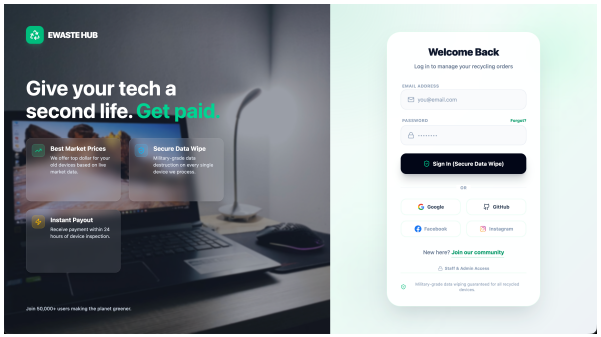

Figure 10: Login and registration page

A.3 Submitting a Device

After logging in, the user can submit a new e-waste request from the device submission page. The form asks for basic device information, including the device name, category, condition, approximate age and market demand level. This information helps the system create a device request and support the classification process.

The user should check that the device information is accurate before submitting the form. Once submitted, the request is stored in the system and linked to the user's account.

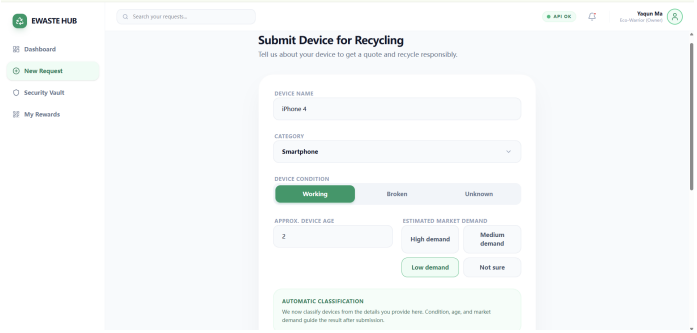

Figure 11: Device submission form

A.4 Viewing Classification Results

After a device has been submitted, the system shows a classification-related result. The result may indicate whether the device is current, recyclable, rare, unknown or unwanted. Each classification gives the user a different next step.

Classification	Meaning for the user
Current	The device may still have resale or referral value.
Recycle	The device should be processed for recycling, with possible data retrieval or wiping options.
Rare	The device may need special review because it could have collectable or unusual value.
Unknown	The system cannot classify the device confidently, so staff review is required.
Unwanted	The device is not suitable for the main resale or retrieval workflow.

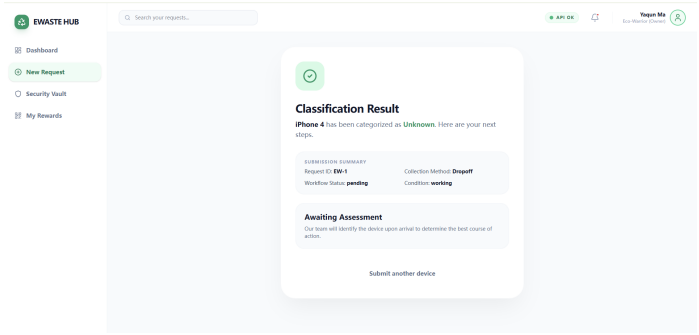

Figure 12: Device classification result

A.5 Tracking Submitted Devices

Users can open the dashboard to view their submitted devices. The dashboard shows important information such as device name, category, status, processing state, market demand and device age. Users can select a device to view more detailed information about the request.

This page helps users understand whether their device is still waiting for review, has been processed, or needs further action.

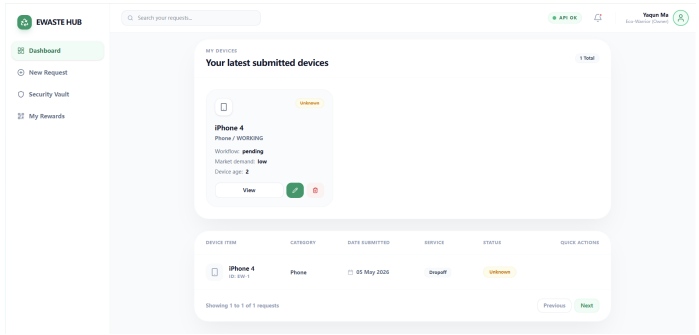

Figure 13: User dashboard and submitted devices

A.6 Requesting Data Retrieval

For recyclable devices, users may need to retrieve important files before the device is wiped or recycled. The system includes a data retrieval workflow foundation. Users can follow the available retrieval option, review the related information and proceed through the request process where available.

In the final prototype, this function demonstrates the intended retrieval workflow. Production email delivery, cloud storage delivery and full external payment verification are treated as future improvements.

A.7 Logging Out

After finishing their session, users should log out from the account menu. Logging out clears the local login session and returns the user to the login page. This is especially important when using a shared computer during assessment or demonstration.

A.8 Summary of Main User Steps

Step	Action	Purpose
1	Open the website	Access the eWaste Hub homepage.
2	Register or log in	Enter the protected user area.
3	Submit a device	Provide device details for classification.
4	View the result	Understand whether the device should be resold, recycled, reviewed or treated differently.
5	Check the dashboard	Track submitted devices and request status.
6	Log out	End the session safely.

Appendix B. Setup Guide

This appendix provides a setup guide for non-developer clients or assessors who want to run the eWaste Hub from a clean copy of the project repository. It covers the required software, clean repository download, installation, configuration, local running steps and optional public demo access.

B.1 Required Software

Before running the project, the computer should have the following software installed:

Software	Purpose
Git	Downloads and updates the project repository from GitLab/GitHub.
Python 3.11 or later	Runs the Flask backend API and setup scripts.
Node.js 20 or later	Installs and runs the React/Vite frontend.
Modern web browser	Opens the local eWaste Hub web application. Chrome or Edge is recommended.
ngrok, optional	Creates a temporary public demo link so another person can open the local project remotely.

B.2 Clean Repository Download

A new user should first obtain a clean copy of the project repository. If Git is available, the recommended method is:

```
git clone <repository-url>
cd project
git pull
```

If the project has already been downloaded before, the user can update it with:

```
git pull
```

The local runtime database file is not normally committed to the repository. Instead, the launcher creates and prepares the local SQLite database automatically. This avoids sharing personal runtime data while still allowing every new user to start the project from the same source code and setup scripts.

B.3 Local Configuration

The repository includes example environment files for the frontend and backend. For local assessment or demonstration, the default settings are usually enough because the launcher sets the local database path and starts both services automatically.

If manual configuration is required, the user can copy the example environment files and edit the values:

```
backend/.env.example -> backend/.env
frontend/.env.example -> frontend/.env
```

For a normal local run, the backend uses a SQLite database in the backend instance folder. The project uses one shared database path for the Flask backend and supporting database bridge scripts, so the frontend, backend and staff/admin pages read from the same local data source.

B.4 Running the Project on Windows

For a non-technical Windows user, the easiest method is to run the launcher:

```
launcher/run.bat
```

The launcher performs the main startup tasks automatically. It checks the required environment, installs or verifies dependencies where needed, starts the Flask backend service, starts the Vite frontend service, prepares the local database and opens the application in the browser.

By default, the local services use these addresses:

Service	Local address
Frontend	http://127.0.0.1:5173
Backend API	http://127.0.0.1:5050

If the browser does not open automatically, the user can manually open http://127.0.0.1:5173.

B.5 Demo Accounts

The launcher prepares demo accounts for local testing. These accounts allow assessors to test the three main roles without manually creating users first.

Role	Example email	Purpose
Consumer	user01@ewastehub.demo	Tests the owner device submission and dashboard workflow.
Staff	staff01@ewastehub.demo	Tests internal staff request, device and unknown queue management.
Admin	admin01@ewastehub.demo	Tests user management, reports, payments and administrator pages.

The shared demo password is:

```
EwasteDemo!2026
```

The system also presents social login options. Facebook and Instagram are supported as demo login flows for presentation purposes. Google and GitHub require external OAuth configuration before they can be used as real third-party login providers.

B.6 Optional Public Demo Link

If the project needs to be shown to another person outside the local computer, the public demo launcher can be used with ngrok:

```
launcher/public_demo.bat
```

This starts the local frontend and backend, then creates a temporary public URL. The generated ngrok address can be shared with teammates, assessors or clients for demonstration. The link is temporary and only works while the local computer and launcher are running.

For first-time ngrok use, the user may need to install ngrok and configure an account authtoken. After setup, the launcher can create a public demo link automatically.

B.7 Common Troubleshooting

Problem	Suggested fix
Browser cannot open the app	Check that the launcher is still running, then open `http://127.0.0.1:5173` manually.
Login says invalid credentials	Run the launcher again so the local database and demo accounts are prepared. Use the demo password exactly as written.
Frontend does not start	Check that Node.js 20 or later is installed.
Backend does not start	Check that Python 3.11 or later is installed.
Public link does not work	Restart `launcher/public_demo.bat` and use the newest ngrok URL. Old tunnel links expire when the process stops.
OAuth login does not redirect correctly	Use email/password or Facebook/Instagram demo login for local assessment unless Google/GitHub OAuth callback URLs have been configured.

B.8 Notes for Deployment

The local launcher is intended for development, assessment and demonstration. For production deployment, the application should be deployed with separate frontend and backend hosting, secure environment variables, a managed database, HTTPS, configured OAuth callback URLs, and production-ready payment/email/cloud service credentials.